



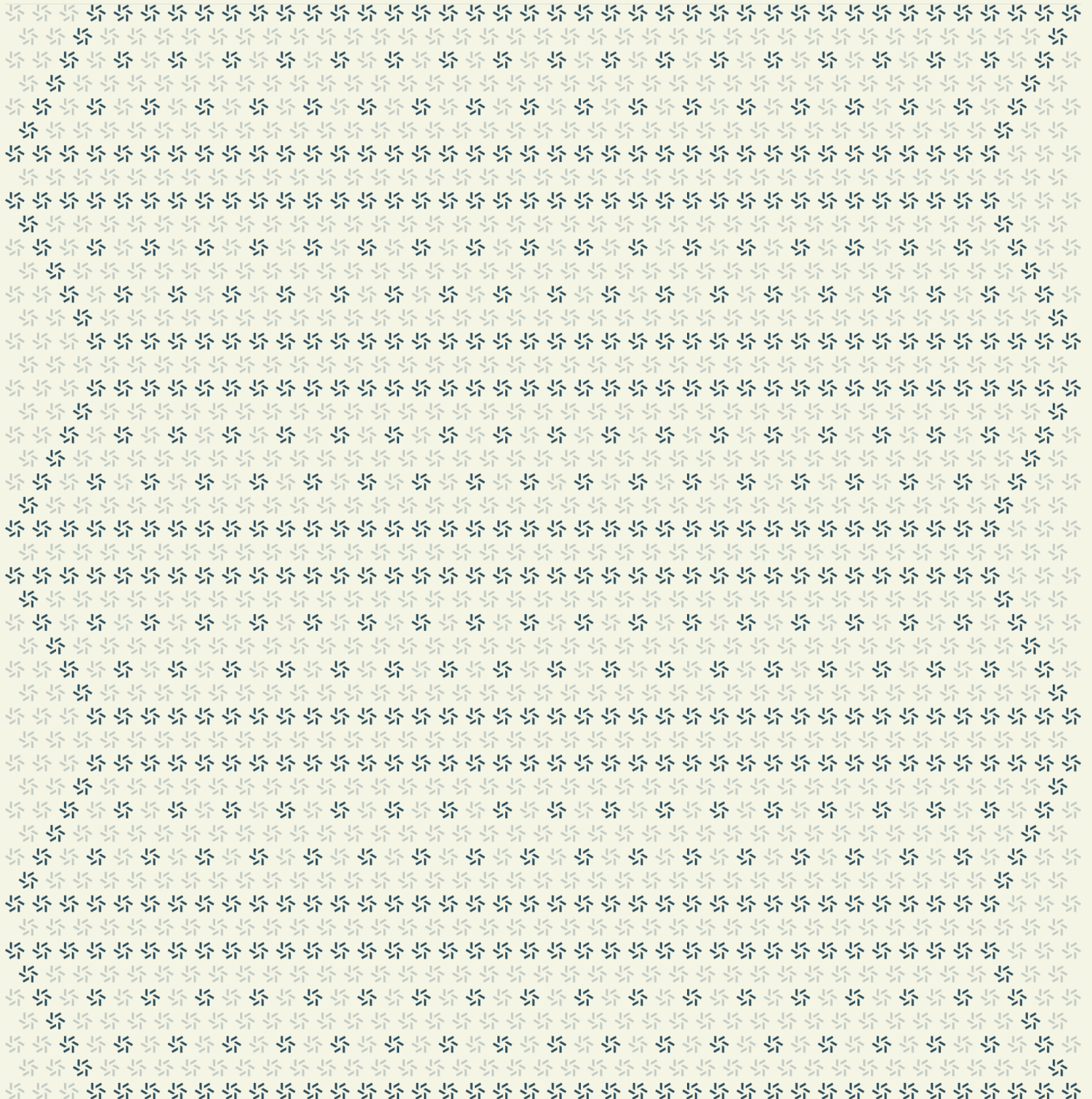
Prepared for
Ryan Zarick
Isaac Zhang
LayerZero Labs

Prepared by
Aaron Esau
Ethan Lee
Zellic

December 4, 2025

OneSig Solana

Solana Application Security Assessment



Contents

About Zellic 4

1. Overview 4

- 1.1. Executive Summary 5
 - 1.2. Goals of the Assessment 5
 - 1.3. Non-goals and Limitations 5
 - 1.4. Results 5
-

2. Introduction 6

- 2.1. About OneSig Solana 7
 - 2.2. Methodology 7
 - 2.3. Scope 9
 - 2.4. Project Overview 9
 - 2.5. Project Timeline 10
-

3. Discussion 10

- 3.1. Missing zero-address checks in `add_executor` and `add_signer` 11
 - 3.2. Inconsistent operator in Merkle-root expiry checks 11
 - 3.3. Test suite 12
-

4. Threat Model 12

- 4.1. Program: `onesig` 13
-

5.	Assessment Results	20
5.1.	Disclaimer	21

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for LayerZero Labs from November 28th to December 3rd, 2025. During this engagement, Zellic reviewed OneSig Solana's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can transactions be executed without valid signatures?
 - Can Merkle-root or proof validation be bypassed?
 - Is the Solana implementation consistent with the EVM implementation?
 - Can executor/signer configuration be manipulated?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped OneSig Solana programs, there were no security vulnerabilities discovered.

Zellic recorded its notes and observations from the assessment for the benefit of LayerZero Labs in the Discussion section ([3.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
 Critical	0
 High	0
 Medium	0
 Low	0
 Informational	0

2. Introduction

2.1. About OneSig Solana

LayerZero Labs contributed the following description of OneSig Solana:

Solana OneSig is a smart contract adapted from the EVM OneSig design, enabling permissionless execution of pre-authorized transactions through Merkle tree signatures. The design goal is to reduce execution friction, eliminate repetitive on-chain approvals.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped programs itself. These observations — found in the Discussion (3.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

OneSig Solana Programs

Type	Rust
Platform	SVM-compatible
Target	onesig-internal
Repository	https://github.com/LayerZero-Labs/onesig-internal ↗
Version	cc4c1c1487b14ab329456379d68b71dbfb2ff496
Programs	packages/onesig-solana/programs/onesig/**/*.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.1 person-weeks. The assessment was conducted by two consultants over the course of 3.5 calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Aaron Esau
✈ Engineer
aaron@zellic.io ↗

Ethan Lee
✈ Engineer
ethl@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

November 28, 2025 Start of primary review period

December 3, 2025 End of primary review period

3. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

3.1. Missing zero-address checks in `add_executor` and `add_signer`

A small detail that causes the EVM and Solana versions to differ is that `_addExecutor` and `_addSigner` in the EVM version have a zero-address check, but `add_executor` and `add_signer` in the Solana version lack that check. This has no impact, but we recommend adding the checks to be consistent.

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in [PR #345](#). They intend to merge these changes into the production branch.

3.2. Inconsistent operator in Merkle-root expiry checks

In `validation/merkle.rs`, the Merkle-root expiry check ensures `expiry >= current_timestamp`:

```
impl MerkleValidator {
    /// Verifies Merkle root expiry and signatures
    pub fn verify_merkle_root(
        one_sig_state: &OneSigState,
        merkle_root: &Hash,
        expiry: i64,
        signatures: &[u8],
        current_timestamp: i64,
    ) -> Result<()> {

        require!(expiry >= current_timestamp, OneSigError::ExpiredMerkleRoot);
    }
}
```

However, in `instructions/execute_transaction.rs`, it ensures `expiry > current_timestamp`:

```
#[event_cpi]
#[derive(Accounts)]
pub struct ExecuteTransaction<'info> {
    pub executor: Signer<'info>,
    /// CHECK: This is the same PDA used in invoke_signed when executing
    transactions.
    /// It signs on behalf of the program in execute_transaction.
    #[account(seeds = [ONE_SIG_SEED, one_sig_state.key().as_ref()],
```

```
bump = one_sig_state.bump)]  
pub one_sig_signer: AccountInfo<'info>,  
#[account(mut)]  
pub one_sig_state: Account<'info, OneSigState>,  
#[account(  
    seeds = [MERKLE_ROOT_SEED, one_sig_state.key().as_ref(),  
    merkle_root_state.merkle_root.as_ref()],  
    bump = merkle_root_state.bump,  
    constraint = merkle_root_state.expiry > Clock::get()?.unix_timestamp  
        @OneSigError::ExpiredMerkleRoot,  
    constraint = merkle_root_state.seed == one_sig_state.seed  
        @OneSigError::SeedMismatch,  
    )]  
pub merkle_root_state: Option<Account<'info, MerkleRootState>>,  
}
```

Similarly, a constraint intended to check that the Merkle root has already expired exists in `instructions/close_merkle_root.rs` and uses the `< sign` (for which the logical opposite would be the `>= sign`):

```
constraint = merkle_root_state.expiry < Clock::get()?.unix_timestamp  
    @OneSigError::MerkleRootNotExpired)
```

Though this has no impact, we recommend standardizing one approach to be consistent. Note that the EVM version reverts if `current_timestamp > expiry`.

This issue has been acknowledged by LayerZero Labs, and a fix was implemented in [PR #345](#). They intend to merge these changes into the production branch.

3.3. Test suite

The test suite covers success scenarios for all instructions and most common error cases. However, several error paths remain untested; `ExecuteTransaction` covers only six of 14 possible errors (missing `Reentrancy`, `ExcessiveBalanceDeduction`, and `InvalidSignerOwner`, etc.), and `CloseMerkleRoot` does not test the `Unauthorized` error. We recommend expanding the test suite to cover these missing error cases.

4. Threat Model

As time permitted, we analyzed each instruction in the program and created a written threat model for the most critical instructions. A threat model documents the high level functionality of a given instruction, the inputs it receives, and the accounts it operates on as well as the main checks performed on them; it gives an overview of the attack surface of the programs and of the level of control an attacker has over the inputs of critical instructions.

For brevity, system accounts and well known program accounts have not been included in the list of accounts received by an instruction; the instructions that receive these accounts make use of Anchor types which automatically ensure that the public key of the account is correct.

Discriminant checks, ownership checks and rent checks are not discussed for each individual account; unless otherwise stated, the program uses Anchor types which perform the necessary checks automatically.

Not all instructions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that an instruction is safe.

4.1. Program: onesig

Instruction: InitOneSig

This instruction initializes a new one-sig state account.

Input structure

```
pub struct InitOneSigParams {
    pub one_sig_id: u64, // The unique identifier for the one-sig state account
    pub seed: Hash, // The random seed for the signatures and Merkle root
    pub threshold: u8, // The threshold of signers required to execute a
    transaction
    pub signers: Vec<Address>, // The signers who are authorized to sign
    transactions
    pub executors: Vec<Pubkey>, // The executors who are authorized to execute
    transactions
    pub executor_required: bool, // Whether executor permission is required
    for transaction execution
}
```

Accounts

- **payer:** The account that pays for the one-sig state-account creation.
- **state:** The one-sig state account that is being initialized.

- Signer: No.
- Init: Yes.
- PDA: No.
- Writable: Yes.
- Other constraints: None.
- **system_program**: The Solana System Program account.

Test coverage

Only basic initialization tests exist via the `setupOneSig` helper function used in test setup. There are no error scenarios or edge cases.

Instruction: SetConfig

This instruction is used to update the configuration of a one-sig state account.

Input structure

```
pub enum SetConfigParams {
    AddSigner(Address),           // Adds a signer to multi-sig
    RemoveSigner(Address),        // Removes a signer from multi-sig
    SetThreshold(u8),             // Sets the signature threshold for multi-sig
    SetSeed(Hash),                // Updates the seed value to invalidate
    previously signed transactions
    AddExecutor(Pubkey),          // Adds an executor to executors
    RemoveExecutor(Pubkey),       // Removes an executor from executors
    SetExecutorRequired(bool),    // Sets whether executor permission is
    required for transaction execution
}
```

Accounts

- **one_sig_signer**: The account that is authorized to update the configuration of the one-sig state account.
 - Signer: Yes.
 - Init: No.
 - PDA: Yes, with `ONE_SIG_SEED` and `state.key()` as seeds.
 - Writable: No.
 - Other constraints: None.

- **state:** The one-sig state account that is being updated.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Other constraints: None.

Additional checks/behavior

- **AddSigner:** The signer's count must be $< \text{SIGNERS_MAX_LEN}$ (20), and the signer must not already exist.
- **RemoveSigner:** The signer must exist, and the remaining signer's count must be $\geq$ threshold after removal.
- **SetThreshold:** The threshold must be > 0 , $\leq \text{MAX_THRESHOLD}$ (13), and $\leq$ current signer's count.
- **SetSeed:** No validation — directly updates the seed value.
- **AddExecutor:** The executor's count must be $< \text{EXECUTORS_MAX_LEN}$ (277), and the executor must not already exist.
- **RemoveExecutor:** The executor must exist, and if `executor_required` is true, at least one executor must remain.
- **SetExecutorRequired:** If setting to true, the executor's set must be nonempty.

Test coverage

- All seven `SetConfigParams` variants are covered by success tests.
- Eight of nine possible errors are covered. However, `InvalidExecutorsLen` and `EmptyExecutorSet` errors for the `SetExecutorRequired` configuration are not covered.
- The test should verify `MAX_THRESHOLD` is constrained by transaction size limit validates that `MAX_THRESHOLD` (13) is properly limited by Solana's transaction size constraints.

Instruction: `VerifyMerkleRoot`

This instruction is used to verify a Merkle root and store the verified Merkle root in a Merkle-root state account.

Input structure

```
pub struct VerifyMerkleRootParams {  
    pub merkle_root: Hash, // Expected Merkle root  
    pub expiry: i64, // Root validity timestamp  
    pub signatures: Vec<u8>, // Concatenated signatures  
}
```

The expiry must be greater than the current timestamp.

Accounts

- **payer**: The account that pays for the Merkle-root state-account creation.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Other constraints: None.
- **merkle_root_state**: The Merkle-root state account that stores the verified Merkle root.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, with MERKLE_ROOT_SEED, `one_sig_state.key()`, and `params.merkle_root` as seeds.
 - Writable: Yes.
 - Other constraints: None.
- **one_sig_state**: The one-sig state account containing multi-sig configuration.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Other constraints: None.
- **system_program**: The Solana System Program account.

Additional checks/behavior

- The threshold of the one-sig state account must be greater than zero and less than or equal to the number of signatures.
- The number of signatures must be a multiple of the signature length (65 bytes) and must be greater than or equal to the threshold.

Test coverage

- The success scenario is covered.
- Five signature-validation errors are covered (ExpiredMerkleRoot, InsufficientSignatures, MissingSigner, DuplicateSigners, and FailedSignatureRecovery) via one-step execution tests.
- The SignatureDataSizeMismatch error is not covered.

Instruction: ExecuteTransaction

This instruction executes a verified transaction via a Merkle proof with either one-step (direct signature verification) or two-step (preverified Merkle-root state) execution mode.

Input structure

```
pub struct ExecuteTransactionParams {  
  pub transaction: OneSigTransaction, // Transaction data with Merkle proof  
  pub merkle_root_verification: Option<VerifyMerkleRootParams>, // Optional  
  // for one-step execution  
}  
  
pub struct OneSigTransaction {  
  pub ix_data: Vec<u8>, // Instruction data to execute  
  pub value: u64, // Maximum SOL that can be spent  
  pub proof: Vec<Hash>, // Merkle proof  
}  
  
pub struct VerifyMerkleRootParams {  
  pub merkle_root: Hash, // Expected Merkle root  
  pub expiry: i64, // Validity timestamp  
  pub signatures: Vec<u8>, // Concatenated signatures  
}
```

Accounts

- **executor:** The account submitting the transaction.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: No.
 - Other constraints: None.

- **one_sig_signer:** The PDA that signs the transaction on behalf of the one-sig state account.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with ONE_SIG_SEED and `one_sig_state.key()` as seeds.
 - Writable: No.
 - Other constraints: None.
- **one_sig_state:** The one-sig state account that contains the multi-sig configuration.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Other constraints: None.
- **merkle_root_state:** Optional preverified Merkle-root state that contains the verified Merkle root for two-step execution.
 - Signer: No.
 - Init: No.
 - PDA: Yes, with MERKLE_ROOT_SEED, `one_sig_state.key()`, and `merkle_root` as seeds.
 - Writable: No.
 - Other constraints: Must not be expired and its seed must match `one_sig_state.seed`.

Additional checks/behavior

- If `executor_required` is true, the executor must be in the executor's list.
- The Merkle root must not be expired.
- If there is no in-line verification, `merkle_root_state` must be provided.
- The Merkle proof must be valid for the encoded leaf.
- This instruction cannot be called recursively.
- The balance change must not exceed the declared value.
- The `one_sig_signer` account must be owned by the system program after execution.
- The `one_sig_signer` account must have empty data after execution.

CPI

It executes an arbitrary instruction via `invoke_signed` with a PDA signature. The first remaining account is the target program ID.

Test coverage

- Success scenarios are covered.
- Six of 14 possible errors are covered via tests, ExpiredMerkleRoot, InsufficientSignatures, MissingSigner, DuplicateSigners, FailedSignatureRecovery, InvalidProof, and SeedMismatch.
- The test should prevent transaction replay attacks validates nonce increment as an edge case.
- Errors not covered include ExecutorRequired, MissingMerkleRootState, Reentrancy, ExcessiveBalanceDeduction, InvalidSignerOwner, NonEmptySignerData, and SignatureDataSizeMismatch.

Instruction: CloseMerkleRoot

This instruction closes a Merkle-root state account after it has expired, returning the rent to the original payer.

Input structure

No input parameters are required.

Accounts

- **signer**: The account that receives the rent refund.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Writable: Yes.
 - Other constraints: Must be the rent payer of the Merkle-root state account.
- **merkle_root_state**: The Merkle-root state account to be closed.
 - Signer: No.
 - Init: No.
 - PDA: Yes.
 - Writable: Yes.
 - Other constraints: Must be expired.

Test coverage

- The success scenario is covered.
- One of two possible errors are covered: MerkleRootNotExpired via should fail to

close merkle root account before expiry.

- The Unauthorized error is not covered (there is no test for a non-rent payer attempting to close).

5. Assessment Results

During our assessment on the scoped OneSig Solana programs, there were no security vulnerabilities discovered.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.